

Extending the Actor Model with Synchronous Delivery: **fast_send** from CPU Threads to FPGA Pipelines

Vincent Mayeski
M2 Tech, Montreal
mayeski@gmail.com

October 2026

Abstract

The actor model has one communication primitive, asynchronous send. Practical actor systems add synchronous request/reply on top of it, as a library pattern (Erlang’s `gen_server:call`, Akka’s `ask`) or as a call restricted to a single scheduling unit (ABS cogs, E vats). We add a second primitive to the model itself: **fast_send**, in which the receiver’s turn runs now, the caller waits for it, and the reply comes back as a value. We give **fast_send** an operational semantics by extending the active-object calculus of De Koster and De Meuter with explicit execution resources, and we compare it with their isolated turn principle, which has a safety half (a turn is isolated and its result is determined) and a liveness half (a turn never blocks). **fast_send** keeps isolation: no two turns of an actor overlap and no turn touches another actor’s state. It weakens determinism to determinism relative to the replies a turn receives. It gives up unconditional liveness: we prove that a turn fails to complete exactly when its thread reaches a cycle in a wait-for graph, and that with asynchronous send alone the graph is empty, which recovers the original result. A per-thread call-chain check sees only the cycles that stay on one thread. We show that **fast_send** is not reducible to asynchronous send followed by a blocking wait once execution resources are shared, which is where the classic encodings of synchronous into asynchronous communication do not apply. For debugging and backtesting, we prove that running every actor — whatever its language or substrate — from one global queue is sound: each execution of that simulator is an execution of the concurrent system, **fast_send** included, so a failure seen in simulation is real, while some failures of the concurrent system, cross-thread deadlock among them, cannot appear in it. Finally we map `send` and **fast_send** onto CPU threads, an FPGA actor runtime, a Rust runtime and, as a design, a GPU runtime: on each substrate the two primitives correspond to two distinct mechanisms, a queue and an inline or blocking call. We report their measured cost on the CPU runtime.

1 Introduction

An actor owns private state and communicates only by messages. In the classic formulations [7, 1, 2] there is exactly one way to communicate: an asynchronous send, which places a message in the receiver’s mailbox and returns at once. The receiver processes its messages one at a time, each in a *turn*. Because a turn never waits on anything, turns cannot deadlock, and because no state is shared, turns cannot race. De Koster and De Meuter name this the *isolated turn principle*, give an operational semantics for four families of actor languages, and prove the principle for each [6].

Real actor systems nonetheless need request/reply, and they obtain it in one of two ways. Either the sender sends a request and then blocks until a reply message arrives — the receiver still runs on its own thread (ABCL/1’s *now*-type messages [13], ABS calls between cogs [9], Erlang’s

`gen_server:call`, Akka’s `ask`) — or a synchronous call is allowed only between objects that already share one scheduling unit (ABS calls inside a cog, E’s immediate calls inside a vat [11]).

This paper studies a third form, which we call **fast_send** and which the kaspar-hft framework provides [10]: the receiver’s turn is run *now*, by an execution resource that the caller is willing to wait on — on a CPU, the caller’s own thread — under the receiver’s exclusive access, and its result is returned to the caller as a value. It is neither a blocking wait on another thread’s work nor a call confined to one scheduling unit. We argue that it belongs in the model rather than in a library, for two reasons. First, its meaning can be stated precisely and its effect on the model’s guarantees characterised exactly (Sections 4 and 5). Second, it maps onto a distinct physical mechanism on every substrate we have built, just as asynchronous send does (Section 7).

Contributions.

1. A thread-aware extension of the active-object calculus of [6] with a **fast_send** rule, in two variants: the callee’s turn runs on the caller’s resource (FAST-LOCAL) or on the callee’s own resource while the caller waits (FAST-REMOTE) (Section 4).
2. Its properties, stated against the isolated turn principle (Section 5): isolation is kept (Theorem 5); determinism holds relative to replies (Theorem 6); liveness holds exactly when no thread reaches a cycle of waits (Theorem 9), and the original liveness result is the special case without **fast_send** (Corollary 10); a per-thread call-chain check detects exactly the cycles confined to one thread (Proposition 11); **fast_send** overtakes queued messages (Proposition 12); and **fast_send** is not send-then-wait when resources are shared (Proposition 13).
3. A soundness theorem for debugging and backtesting from a single global queue, across languages and substrates (Theorem 15), its limits (Proposition 17), and an offline check for the deadlocks it cannot exhibit (Proposition 18) (Section 6).
4. A mapping of both primitives onto CPU threads, an FPGA actor runtime, a Rust runtime and a GPU runtime design, and between them (Section 7), with measurements of the CPU runtime (Section 9).

2 Background: turns and the isolated turn principle

De Koster and De Meuter classify actor languages into four families — classic actors, active objects, processes and communicating event loops — and give each a small-step operational semantics [6]. We build on their active-object calculus, because it matches the programming model of the runtimes studied here: an actor has a fixed interface, a FIFO mailbox and imperative private state.

In that calculus a configuration is a set of actors $\mathcal{A}(\iota_a, \mu, e, O, o)$: an identifier, a mailbox μ of messages $\mathcal{M}\langle msg, \bar{v} \rangle$, the expression e the actor is evaluating, a heap O of passive objects it owns, and an active object o that fixes its interface. Actor-local rules (LET, NEW, INVOKE, FIELD-ACCESS, FIELD-UPDATE, SELF-SEND, RECEIVE) reduce one actor; actor-global rules (SEND, SPAWN) involve two. SEND appends a message to the receiver’s mailbox after deep-copying the passive objects it references (the function `pass`), so no state is shared. RECEIVE starts a turn: when an actor’s expression is a value, the head of its mailbox is removed and the matching method body becomes its expression. A *turn* is the sequence of reductions of one actor between two applications of RECEIVE.

The isolated turn principle then consists of two theorems [6]. *Safety*: the value a turn reduces to is determined up to α -equivalence, whatever the interleaving with other actors, and a turn is not interleaved with another turn of the same actor. *Liveness*: a turn never blocks — every reducible

expression of a turn can take a step — so turns are free of deadlock. The calculi contain no blocking operation; this is what makes liveness hold.

Two features of this setting matter below. The calculus has no notion of the thread or processor that performs a reduction: each actor reduces independently, as if it had its own. And every communication is a `SEND`. A synchronous call needs the first to be made explicit and adds a second kind of communication.

3 Synchronous communication in actor systems

Three reasons are usually given for leaving synchronous calls out of the actor model. *Order*: in Hewitt’s model messages are guaranteed to arrive but in no particular order, so “handle this now, before what I do next” has no meaning. *Deadlock*: an actor that handles one message at a time and waits inside a turn can wait on itself, directly or through others. *Distribution*: across machines, a synchronous call can wait forever on a failed peer.

Where systems provide synchronous communication anyway, they do so in one of two ways.

Wait on the receiver’s thread. ABCL/1 distinguishes *past* (asynchronous), *now* (send and wait for the reply) and *future* messages [13]. In ABS [9], a synchronous call $v = o.m(e)$ between objects in different cogs abbreviates $x = o!m(e)$; $v = x.get$: an asynchronous call followed by a blocking `get`, which “blocks all execution in the object until the return value is available”. Erlang’s `gen_server:call` and Akka’s `ask` are library forms of the same pattern, usually with a timeout. In all of these the receiver’s turn runs on the receiver’s own thread; the caller’s thread waits.

Call within one scheduling unit. In ABS, a synchronous call between objects in the same cog has “the reentrant semantics known from, e.g., Java threads” [9]: control passes to the callee on the cog’s single processor. In E, immediate calls are allowed only on near references, to objects in the same vat; between vats only eventual sends exist [11]. These calls run on the caller’s processor, but they never cross a scheduling unit, so they need no mutual exclusion; and ABS allows re-entry into an object that is already executing.

Expressiveness. Honda and Tokoro [8] and Boudol [4] showed that synchronous communication can be encoded in the asynchronous π -calculus. `fast_send` therefore adds nothing to what can be computed. The encodings assume, however, that every process makes progress independently — that execution resources are unbounded. Proposition 13 shows that once resources are shared, as in an actor group or a cog, send-then-wait deadlocks where `fast_send` completes. The difference between the primitives is a difference in who executes a turn and when.

Where `fast_send` sits. `fast_send` runs the receiver’s turn on a resource the caller is willing to wait on — on a CPU, the caller’s own thread — across scheduling units, under the receiver’s mutual exclusion, and without re-entry: an actor already in a turn cannot be entered again, and a call that tries waits, possibly forever. It combines the “run it here” of the second group with the “any actor” of the first, and pays for it with the deadlock case that the first group has and the second avoids. The rest of the paper makes this precise.

4 A thread-aware calculus with **fast_send**

We extend the active-object calculus of [6] in three ways: execution resources become explicit, an actor's current expression moves from the actor into a *frame* on a resource's stack, and a **fast_send** expression is added. We call the resources *threads*; on an FPGA they are hardware processes (Section 7).

4.1 Semantic entities and syntax

$$\begin{aligned} K \in \text{Configuration} &::= \langle A, T \rangle \\ a \in A \subseteq \text{Actor} &::= \mathcal{A}\langle \iota_a, \mu, O, o, \theta \rangle \\ t \in T \subseteq \text{Thread} &::= \mathcal{T}\langle \theta, S \rangle \quad S ::= \epsilon \mid S \cdot f \\ f \in \text{Frame} &::= \mathcal{F}\langle \iota_a, e, \kappa \rangle \quad \kappa ::= \text{turn} \mid \text{ret}(\rho) \end{aligned}$$

An actor has an identifier, a FIFO mailbox μ , a heap O , an active object o , and a *home thread* θ . Several actors may share a home thread; that is an actor group. A thread has an identifier and a stack S of frames, top on the right. A frame records which actor it belongs to, the expression being evaluated, and why it exists: **turn** for a turn started from the mailbox, or **ret**(ρ) for a turn started by the **fast_send** call ρ , whose value must be returned. Call identifiers ρ are fresh.

The expression syntax of the active-object calculus is extended with one form, and the run-time syntax with one more:

$$e ::= \dots \mid \text{fastsend}(e, \text{msg}, \bar{e}) \quad e ::= \dots \mid \text{wait}(\rho) \quad (\text{run time})$$

Evaluation contexts $\mathcal{E}$ extend the originals left to right through the arguments of **fastsend**. No rule reduces **wait**(ρ): a frame whose expression has the form $\mathcal{E}[\text{wait}(\rho)]$ is *waiting*.

Definition 1 (Busy). Actor ι_a is *busy* in K , written $\text{busy}_K(\iota_a)$, if some thread of K has a frame of ι_a on its stack.

For an actor with active object $o = \mathcal{O}\langle \iota_o, \text{cls}, \bar{v}_f \rangle$ and a message $\mathcal{M}\langle \text{msg}, \bar{v} \rangle$, let **body**($a, \text{msg}, \bar{v}$) be the body of method msg of cls with fields, parameters and **self** substituted, exactly as in **RECEIVE** and **INVOKE** of [6].

4.2 Reduction rules

The actor-local rules of [6] apply unchanged to the *top* frame of any thread, reading and writing the heap of the frame's actor. **SEND** and **SELF-SEND** likewise apply to a top frame and append to the receiver's mailbox. **SPAWN** creates an actor with an empty mailbox and a home thread, either an existing one or a fresh $\mathcal{T}\langle \theta', \epsilon \rangle$. **RECEIVE** and the end of a turn become thread rules, and three rules are new (Figure 1).

- **RECEIVE**: an idle thread starts a turn of one of its idle actors whose mailbox is not empty. With several actors on one thread, the choice among them is free.
- **TURN-END**: a turn started from the mailbox that has reduced to a value is removed. Its value is discarded; a reply to a **send** is an explicit **send**.
- **FAST-LOCAL**: the caller's frame becomes waiting, and the callee's turn is pushed on the *caller's* stack. This is **fast_send** on one CPU thread, or inside one FPGA process.

$$\begin{array}{c}
\frac{t = \mathcal{T}\langle\theta, \epsilon\rangle \quad a = \mathcal{A}\langle\iota_a, \mathcal{M}\langle msg, \bar{v} \rangle \cdot \mu, O, o, \theta \rangle \quad \neg \text{busy}_K(\iota_a)}{K \longrightarrow K[t \mapsto \mathcal{T}\langle\theta, \mathcal{F}\langle\iota_a, \text{body}(a, msg, \bar{v}), \text{turn}\rangle\rangle][a \mapsto \mathcal{A}\langle\iota_a, \mu, O, o, \theta\rangle]} \text{RECEIVE} \\
\\
\frac{t = \mathcal{T}\langle\theta, S \cdot \mathcal{F}\langle\iota_a, v, \text{turn}\rangle\rangle}{K \longrightarrow K[t \mapsto \mathcal{T}\langle\theta, S\rangle]} \text{TURN-END} \\
\\
\frac{t = \mathcal{T}\langle\theta, S \cdot \mathcal{F}\langle\iota_a, \mathcal{E}[\text{fastsend}(\iota_b, msg, \bar{v})], \kappa\rangle\rangle \quad \neg \text{busy}_K(\iota_b) \quad \bar{v}' = \text{pass}(\bar{v}) \quad \rho \text{ fresh}}{K \longrightarrow K[t \mapsto \mathcal{T}\langle\theta, S \cdot \mathcal{F}\langle\iota_a, \mathcal{E}[\text{wait}(\rho)], \kappa\rangle \cdot \mathcal{F}\langle\iota_b, \text{body}(b, msg, \bar{v}'), \text{ret}(\rho)\rangle\rangle]} \text{FAST-LOCAL} \\
\\
\frac{t = \mathcal{T}\langle\theta, S \cdot \mathcal{F}\langle\iota_a, \mathcal{E}[\text{fastsend}(\iota_b, msg, \bar{v})], \kappa\rangle\rangle \quad b = \mathcal{A}\langle\iota_b, \mu_b, O_b, o_b, \theta_b\rangle \quad \theta_b \neq \theta \quad t_b = \mathcal{T}\langle\theta_b, \epsilon\rangle \quad \neg \text{busy}_K(\iota_b) \quad \bar{v}' = \text{pass}(\bar{v}) \quad \rho \text{ fresh}}{K \longrightarrow K[t \mapsto \mathcal{T}\langle\theta, S \cdot \mathcal{F}\langle\iota_a, \mathcal{E}[\text{wait}(\rho)], \kappa\rangle\rangle][t_b \mapsto \mathcal{T}\langle\theta_b, \mathcal{F}\langle\iota_b, \text{body}(b, msg, \bar{v}'), \text{ret}(\rho)\rangle\rangle]} \text{FAST-REMOTE} \\
\\
\frac{t' = \mathcal{T}\langle\theta', S' \cdot \mathcal{F}\langle\iota_b, v, \text{ret}(\rho)\rangle\rangle \quad t = \mathcal{T}\langle\theta, S \cdot \mathcal{F}\langle\iota_a, \mathcal{E}[\text{wait}(\rho)], \kappa\rangle \cdot S''\rangle \quad v' = \text{pass}(v)}{K \longrightarrow K[t' \mapsto \mathcal{T}\langle\theta', S'\rangle][t \mapsto \mathcal{T}\langle\theta, S \cdot \mathcal{F}\langle\iota_a, \mathcal{E}[v'], \kappa\rangle \cdot S''\rangle]} \text{FAST-RETURN}
\end{array}$$

Figure 1: Thread rules. $K[t \mapsto t']$ replaces thread t by t' (and likewise for actors). In FAST-RETURN, t and t' may be the same thread, in which case S'' is empty and the returned frame was directly above the waiting one.

- FAST-REMOTE: the caller's frame becomes waiting, and the callee's turn starts on the callee's own, idle, home thread. This is **fast_send** to an actor on another resource that the caller waits on: a CPU thread calling into an FPGA process, an FPGA process calling a CPU actor, or a call to another machine.
- FAST-RETURN: a callee turn that has reduced to a value is removed and its value, copied, replaces the $\text{wait}(\rho)$ of its caller.

Both **fast_send** rules require the callee to be idle. When it is busy, neither applies: the calling frame is *blocked*. The callee's mailbox is not consulted. A system may use FAST-LOCAL for some pairs of actors and FAST-REMOTE for others; every result below holds for any such mixture.

Remark 2 (Receiver transparency). A method body cannot tell how its turn started: RECEIVE, FAST-LOCAL and FAST-REMOTE all install the same $\text{body}(b, msg, \bar{v})$. This is the calculus counterpart of receiver transparency in kaspar-hft [10]. It is a property of the interface a turn is given, not a claim about what code in a real language can observe about its execution context.

5 Properties

Throughout, $K_0 \longrightarrow^* K$ ranges over configurations reachable from an initial configuration with one actor, as in [6], whose single thread holds no frames.

5.1 Isolation

Lemma 3 (One frame per actor). *In every reachable configuration, each actor has at most one frame, and every frame reads and writes only the heap of its own actor.*

Proof. Initially there are no frames. Only RECEIVE, FAST-LOCAL and FAST-REMOTE create frames, and each creates a frame for an actor that is not busy, that is, has no frame. TURN-END and FAST-RETURN only remove frames; the other rules rewrite the expression of an existing top frame. The heap discipline is that of the active-object calculus: field access and update apply to the heap of the frame's actor, and values crossing actors (SEND, the two **fast_send** rules, FAST-RETURN) are copied by **pass**. $\square$

Definition 4 (Turn). A turn of ι_a is the sequence of reductions from the creation of a frame of ι_a (by RECEIVE, FAST-LOCAL or FAST-REMOTE) to its removal (by TURN-END or FAST-RETURN).

Theorem 5 (Isolation). *No two turns of the same actor overlap, and during a turn of ι_a the heap of ι_a is changed only by reductions of that turn's frame.*

Proof. Immediate from Lemma 3: a second turn would need a second frame. $\square$

This is the safety half of the isolated turn principle, and it holds with **fast_send**. A turn that makes a **fast_send** is suspended while the callee runs, but no other turn of the same actor can run in the meantime, and the callee cannot touch the caller's state.

5.2 Determinism relative to replies

In [6] a turn's value is determined, because a turn receives no input after it starts. With **fast_send** it does: each FAST-RETURN substitutes a value that depends on the callee's state, which other turns may have changed before the call.

Theorem 6 (Determinism relative to replies). *Consider two turns of the same actor that start from α -equivalent heaps and expressions. If the values substituted into them by FAST-RETURN, in order, are α -equivalent, then the two turns end with α -equivalent values and heaps and issue α -equivalent **send** and **fast_send** requests in the same order.*

Proof. As the safety proof of [6], by induction on the reductions of the turn's frame: evaluation contexts fix the order of reduction, and every actor-local rule produces α -equivalent results from α -equivalent inputs. The new cases are the two **fast_send** rules, which rewrite the frame to $\mathcal{E}[\text{wait}(\rho)]$ and perform no reduction on it until FAST-RETURN, whose substituted value is α -equivalent by hypothesis. Reductions of other frames do not touch this turn's heap (Lemma 3). $\square$

A turn that makes no **fast_send** satisfies the original safety theorem unchanged. A turn that does is deterministic as a function of its replies, as it would be if the replies were messages; what **fast_send** removes is the possibility of any other message of the same actor being handled between the request and the reply.

5.3 Liveness

Definition 7 (Blocked frame, wait-for graph). A top frame $\mathcal{F}\langle\iota_a, \mathcal{E}[\text{fastsend}(\iota_b, \text{msg}, \bar{v})], \kappa\rangle$ on thread θ is *blocked* in K if no **fast_send** rule applies to it. The *wait-for graph* W_K has the threads of K as nodes and an edge $\theta \rightarrow \theta'$ when either

- (i) the top frame of θ is blocked on ι_b and θ' holds the frame of ι_b ; or it is blocked by FAST-REMOTE because ι_b 's home thread θ' is not idle; or
- (ii) θ holds a waiting frame $\mathcal{E}[\text{wait}(\rho)]$ whose callee frame $\text{ret}(\rho)$ is on $\theta' \neq \theta$.

Every edge leaves a thread that cannot progress until something on the target thread is removed. A thread whose top frame is waiting has an edge of kind (ii) unless the callee is above it on the same stack, in which case the thread is not stuck: its top frame is the callee's.

Assumption 8 (Terminating handlers, fair scheduling). Every method body terminates when its `fast_send` calls return, and every thread whose top frame can be reduced is eventually reduced.

Theorem 9 (Liveness exactly when waits are acyclic). *Under Assumption 8, a turn in a reachable configuration K fails to complete in some fair continuation of K if and only if, in some configuration reachable from K , its thread has a path in the wait-for graph to a cycle.*

Proof. ($\Leftarrow$) Take the cycle $\theta_1 \rightarrow \dots \rightarrow \theta_n \rightarrow \theta_1$. Each θ_i waits for a frame on θ_{i+1} to be removed. That frame is below the top of θ_{i+1} or is its top; in either case its removal needs θ_{i+1} to progress, which needs a frame on θ_{i+2} removed, and so on around the cycle. No rule removes a frame of a thread whose top frame is blocked or waiting, so none of the θ_i ever progresses, and a thread with a path to the cycle waits on it forever.

($\Rightarrow$) Suppose the thread θ of a turn never completes it, and that no configuration reachable from K has a path from θ to a cycle. Consider a configuration from which θ never again removes a frame. Its outgoing path in W is finite and acyclic, so it ends in a thread θ^* with no outgoing edge: its top frame is neither blocked nor waiting on another thread, so it can be reduced. By Assumption 8 it is reduced, and by termination of method bodies its top frame reaches a value and is removed by `TURN-END` or `FAST-RETURN`. Induction on the number of frames above the one that the predecessor of θ^* waits for shows that this frame is eventually removed, which unblocks the predecessor; repeating along the path, θ progresses — a contradiction. $\square$

Corollary 10 (Liveness without `fast_send`). *If a program never evaluates `fastsend`, every turn completes under Assumption 8.*

Proof. Without `fast_send` there are no blocked or waiting frames, so the wait-for graph has no edges. $\square$

This is the liveness half of the isolated turn principle, recovered as the case without `fast_send`. With `fast_send`, liveness is a property of the program's pattern of synchronous calls rather than of the model.

Proposition 11 (What a per-thread call chain detects). *Suppose a thread checks, before each `FAST-LOCAL` to ι_b , whether ι_b has a frame on its own stack. The check succeeds exactly for the self-loops $\theta \rightarrow \theta$ of the wait-for graph. It does not detect cycles of length two or more.*

Proof. A self-loop arises when the top frame of θ is blocked on an actor whose frame is on θ itself, which is what the check tests. A cycle through two threads $\theta_1 \rightarrow \theta_2 \rightarrow \theta_1$ arises when the top frame of each is blocked on an actor whose frame is on the other: each check inspects only its own stack, finds the callee absent, and passes. For example, thread 1 runs a `fast_send` to A whose body makes a `fast_send` to B , while thread 2 runs a `fast_send` to B whose body makes a `fast_send` to A . $\square$

A per-thread detector, such as the call-chain test designed in [10],¹ therefore turns same-thread cycles into errors and leaves cross-thread cycles as deadlocks. Detecting those needs shared state: a global wait-for graph, a global order on actors, or bounded waiting with a fallback to asynchronous delivery.

¹kaspar-hft does not implement the call-chain test yet: `fast_send` takes the receiver's mutex directly, so a synchronous cycle currently deadlocks.

5.4 Order

Proposition 12 (Order). *(1) Messages sent with `send` from ι_a to ι_b start turns of ι_b in the order they were sent. (2) A `fast_send` from ι_a to ι_b starts its turn before any message already in ι_b ’s mailbox, including earlier `sends` from ι_a . (3) Messages sent during the callee’s turn of a `fast_send` are enqueued before any message the caller sends after the `fast_send` returns.*

Proof. (1) SEND appends and RECEIVE removes the head. (2) The `fast_send` rules start the callee’s turn without consulting its mailbox. (3) The caller’s frame is waiting from the `fast_send` until FAST-RETURN, and the callee’s sends happen before that. $\square$

Part (2) is the formal counterpart of “`fast_send` bypasses the mailbox”. On the FPGA runtime of Section 7 it is implemented by a dedicated input port that each actor process serves before its mailbox FIFO.

5.5 `fast_send` is not send-then-wait

Let *send-then-wait* be the pattern of the first group in Section 3: the caller sends a request with `send` and its frame then blocks until a reply message for it arrives. We model the blocking by a frame that no rule reduces until the reply has been enqueued in the caller’s mailbox; the frame stays on its thread.

Proposition 13 (Shared resources). *Let actors ι_a and ι_b share a home thread, and let ι_b ’s method return without further communication. A turn of ι_a that obtains ι_b ’s result by *send-then-wait* never completes. The same turn using `fast_send` with FAST-LOCAL completes.*

Proof. With send-then-wait, the request is enqueued in ι_b ’s mailbox and ι_a ’s frame stays on the shared thread θ . RECEIVE for ι_b requires θ to have an empty stack, which it never has again, so the reply is never produced. With `fast_send`, FAST-LOCAL pushes ι_b ’s turn on θ above ι_a ’s frame; it terminates and FAST-RETURN resumes ι_a . $\square$

The encodings of synchronous into asynchronous communication [8, 4] are sound in a setting where every process progresses independently. In a setting with a bounded number of execution resources — actor groups, ABS cogs, E vats, a pinned thread per core, a fixed number of FPGA processes — `fast_send` and send-then-wait differ in whether they complete. This is also why systems of the second group in Section 3 permit synchronous calls exactly inside a shared scheduling unit: there, send-then-wait is not an option.

6 Simulation and debugging from a global queue

A backtest replays recorded inputs through the same actors that trade live. A debugger needs the same thing plus the ability to stop, step and inspect. Both are easiest when every actor in the system — whatever language it is written in and whatever substrate it normally runs on — is driven from a single queue by a single thread: every message goes into the queue, receivers take their messages from it, and the execution is one deterministic sequence that can be replayed, paused and inspected. The kaspar-hft actor group, which co-schedules actors behind one mailbox, already serves as such a simulator for CPU actors [10].² The question is whether `fast_send` breaks the correspondence between that simulator and the concurrent system.

²The cross-language, cross-substrate global queue described in this section is not implemented in kaspar-hft yet; CPU actors in one group already run this way.

6.1 The global-queue semantics

The global-queue semantics GQ is the calculus of Section 4 with one thread θ_0 , the home thread of every actor, and with per-actor mailboxes replaced by one FIFO queue Q of pairs (ι_b, m) :

- **SEND** appends $(\iota_b, \mathcal{M}\langle msg, \text{pass}(\bar{v}) \rangle)$ to Q ;
- **RECEIVE-GQ**: when θ_0 has an empty stack, remove the head (ι_b, m) of Q and start ι_b 's turn for m ;
- **fast_send** uses **FAST-LOCAL** on θ_0 .

An actor that normally runs on another substrate takes part through its method bodies alone: an FPGA actor's turn is its step function, executed by the simulator in software; a Rust actor's turn is its handler, called across the language boundary. By receiver transparency, the body is the same whatever starts the turn.

The production semantics P is the calculus of Section 4 with any assignment of actors to threads and any choice of **FAST-LOCAL** or **FAST-REMOTE** per call site, subject to one condition that the runtimes studied here satisfy:

Assumption 14 (Dedicated remote resources). A thread that serves as the home thread of a **FAST-REMOTE** callee is the home of that actor only.

On the FPGA runtime each actor is its own process, so the assumption holds for **fast_send** calls into the FPGA. CPU actors that share a thread are called with **FAST-LOCAL**.

6.2 Soundness

Say that two executions *agree* if every actor starts the same turns in the same order, with α -equivalent messages, and every turn ends with an α -equivalent value and heap.

Theorem 15 (Global-queue simulation is sound). *Under Assumption 14, for every finite execution of GQ from a given initial configuration there is an execution of P from the corresponding initial configuration that agrees with it. In particular, a configuration in which a turn is stuck, or an error state, reached in GQ has an agreeing counterpart in P .*

Proof. We build the P execution step by step, maintaining the invariant that at every point where θ_0 has an empty stack in GQ , every thread of P has an empty stack, the heaps agree, and each actor's mailbox in P is the projection of Q onto that actor, in order.

Receive. GQ takes the head (ι_b, m) of Q . By the invariant, m is the head of ι_b 's mailbox in P , since every earlier pair for ι_b in Q has already been removed. ι_b 's home thread is idle and ι_b is not busy, so **RECEIVE** applies in P to the same message.

Within a turn. P reduces only the thread that holds the corresponding frame, mirroring each GQ step: actor-local steps are identical; **SEND** appends to the receiver's mailbox, which preserves the projection invariant.

fast_send. In GQ , **FAST-LOCAL** to ι_b applies if and only if ι_b has no frame on θ_0 's stack. In the mirrored P execution the only frames in existence are the ones mirroring that stack, so ι_b is busy in P exactly when it is busy in GQ . If P uses **FAST-LOCAL** for the call, it applies. If it uses **FAST-REMOTE**, the rule also needs ι_b 's home thread to be idle. By Assumption 14 a frame reaches that thread only as a turn of ι_b or stacked above one; ι_b has no frame, so the thread is idle. The callee's turn then proceeds as above, and **FAST-RETURN** applies in both.

If GQ is stuck on a **fast_send** to a busy actor, the mirrored P frame is blocked on the same actor. Agreement of values and heaps follows from Theorem 6, since the mirrored turns receive the same replies. $\square$

Corollary 16 (What the simulator shows is real). *Every value a **fast_send** returns in GQ , every assertion failure, error state or same-thread **fast_send** cycle it reaches, can occur in the concurrent system. A GQ run is deterministic given its inputs, so it can be replayed exactly.*

So **fast_send** does not break the global-queue simulation. What makes the argument go through is that in a single-threaded run the callee of a **fast_send** is idle unless it is already on the current call chain: the simulator only ever takes the branch of **fast_send** in which the callee was free, which is one of the branches the concurrent system can take. It is not obvious in general: without Assumption 14, a FAST-REMOTE callee whose resource is occupied by another actor blocks in P where GQ proceeds, and the simulator would then show behaviour the concurrent system cannot produce.

6.3 What the simulator cannot show

Proposition 17 (Incompleteness). *There are executions of P that agree with no execution of GQ . In particular: (1) a deadlock on a wait-for cycle through two or more threads; (2) a turn of ι_a that makes a **fast_send** to ι_b after another thread has run a turn of ι_b that started after ι_a 's turn began.*

Proof. In GQ only one thread exists, so the wait-for graph can contain only self-loops, and no turn starts while another is in progress except as a **fast_send** callee on the same stack. Both executions described require a turn to start on another thread while ι_a 's turn is in progress. $\square$

A backtest can therefore confirm bugs but cannot rule out the two classes above. The first can be approached offline:

Proposition 18 (Cycles in the call graph). *Let G be the directed graph on actors with an edge $\iota_a \rightarrow \iota_b$ whenever some turn of ι_a can evaluate a **fast_send** to ι_b . If G is acyclic, no execution of P reaches a wait-for cycle.*

Proof. The frames on any thread, bottom to top, belong to a path in G , since each frame above the bottom one was pushed by a **fast_send** from the frame below (FAST-LOCAL); a FAST-REMOTE callee's frame extends the same path onto another thread. A wait-for edge $\theta \rightarrow \theta'$ adds the edge from the actor on top of θ to an actor already on the path held by θ' . Following a wait-for cycle and concatenating these paths yields a cycle in G . $\square$

A GQ run can record the **fast_send** edges it executes. The recorded graph is a subgraph of G — it contains only the calls the inputs exercised — so a cycle in it is a real risk to be removed, and the absence of a cycle is evidence only for the paths the backtest covered. A static analysis of the call sites would give all of G .

Time. GQ reproduces what happens, not when. Latency on each substrate is a separate question, answered by measurement or by the substrate's own timing model (Section 9).

Table 1: `send` and `fast_send` on each substrate and between substrates, and the rule of Section 4 each `fast_send` implements.

Where	<code>send</code>	<code>fast_send</code>
CPU, one process	message into the receiver’s mailbox; a thread per actor or per group	callee’s handler runs on the caller’s thread under the callee’s lock (FAST-LOCAL)
C++ $\leftrightarrow$ Rust, one process	message converted and enqueued through a C ABI bridge	callee’s handler runs on the caller’s thread through the bridge (FAST-LOCAL)
FPGA, one process	—	callee is an actor object inside the caller’s process; its handler is inlined into the caller’s pipeline (FAST-LOCAL)
FPGA, between processes	on-chip router into the receiver’s mailbox FIFO	request and reply relayed through the host (FAST-REMOTE)
CPU $\rightarrow$ FPGA	envelope over PCIe, router, mailbox FIFO	envelope to the actor’s <code>fast_send</code> port, served before its mailbox; the CPU thread waits for the reply (FAST-REMOTE)
FPGA $\rightarrow$ CPU	envelope over PCIe; a host bridge thread enqueues it	request over PCIe; the bridge thread runs the CPU handler inline; the FPGA process stalls until the reply returns (FAST-REMOTE)
GPU (design)	ring buffer in device memory	device function call inside one kernel (FAST-LOCAL); host calls wait on a reply ring (FAST-REMOTE)
Another machine	network message	request/reply RPC; the caller waits (FAST-REMOTE)

7 Mapping `send` and `fast_send` onto substrates

The calculus leaves open what a thread is and how a mailbox is built. Each substrate supplies both, and each supplies a distinct mechanism for each primitive (Table 1).

The pattern is the same everywhere. `send` is a queue between two independently scheduled resources. `fast_send` is either an inline call on the caller’s resource, where the substrate lets one resource run another actor’s handler, or a blocking request to a resource dedicated to the callee, where it does not. The calculus of Section 4 covers both with one semantics, and Theorems 5, 6 and 9 hold for any mixture.

A `fast_send` is available between any two substrates where the caller can wait. A CPU thread can wait on a condition. An FPGA process can wait on a blocking stream read, which stalls that process alone while the rest of the chip keeps running. What differs between substrate pairs is the cost of the wait, which Section 9 measures where it can.

8 Implementations

CPU runtime. The kaspar-hft C++ runtime [10] implements actors with a mutex-protected turn, a selectable mailbox per actor, `send`, `fast_send` and groups. `fast_send` takes the callee’s lock on the caller’s thread and runs its handler inline; a reply is returned as a value. A delivery flag is private to the framework, and the two queue fields a handler may read are defined identically for both delivery modes, so the handler receives no information about how it was reached. A mailbox

Table 2: Round trip of one ping/pong with a reply, median over runs, AMD EPYC 9374F. The **fast_send** figure is amortized over 10^6 round trips because its per-sample value is below the clock’s resolution of about 10 ns.

Implementation	ns	What it pays for
send , two threads, sleeping	~ 3370	two cross-core handoffs and two thread wake-ups
send , two threads, busy-polling	370	two cross-core handoffs; each thread holds a core
send , one group (one thread)	90	two queue operations
fast_send	~ 30	two inline calls under the callee’s lock

may also be configured to busy-poll instead of sleeping, which changes the cost of **send** between threads but not its semantics.

Rust runtime and interop. A separate Rust actor runtime hosts Rust actors. C++ and Rust actors reach each other through generated C-ABI bridges, from one message schema; both **send** and **fast_send** work across the boundary.³

FPGA runtime. The FPGA runtime is written in Vitis HLS C++. Each actor is a hardware process with a mailbox FIFO, a **fast_send** port served before the mailbox, and a port on which replies to its own remote **fast_sends** arrive; its handlers are selected by a switch on the message identifier. A router delivers messages by destination actor, using a route table fixed when the design is built, and a host link connects the design to PCIe. On the CPU, an **ActorRef** can refer to an FPGA actor; **send** and **fast_send** on it write envelopes to the card, and a bridge thread delivers messages and **fast_send** requests from the card to CPU actors, passing a stand-in for the FPGA actor as the sender so that ordinary replies return to the chip. The design runs today in software, with one thread per hardware process.⁴

GPU runtime. The same structure maps onto a GPU: actors as warps or thread blocks of a persistent kernel, mailboxes as device-memory rings, **fast_send** inside the kernel as a device function call, and a host bridge over rings in pinned memory. It is a design; we have not built it.

9 Evaluation

CPU. Table 2 gives the round-trip cost of a ping/pong exchange with a reply on an AMD EPYC 9374F (RHEL 9.2, `g++ -O3 -march=native`, two dedicated cores). The four rows are four implementations of the same exchange in the calculus: **send** between two threads that sleep when idle, the same with threads that busy-poll, **send** within one group, and **fast_send**.

Most of the cost of **send** between threads is putting a thread to sleep and waking it; the cross-core handoff itself is about 185 ns each way. **fast_send** costs about one mispredicted virtual call per hop (9.1 ns against 7.5–9.3 ns, measured the same way) and about $5\times$ a direct call [10].

³kaspar-hft does not return **fast_send** replies across the language boundary yet; a cross-language **fast_send** delivers the message and returns no value.

⁴The FPGA runtime has not yet been synthesized or run on a card; **fast_send** between two FPGA processes is relayed through the host.

Across languages. On an Apple Silicon machine, a one-way `fast_send` without a reply costs about 13–15 ns within one language, 46 ns from C++ to Rust and 70 ns from Rust to C++; the difference is the marshalling, the C-ABI call, and a per-call lookup of the receiver by name.

FPGA. The FPGA runtime’s semantics are tested in software: `send` through the router, `fast_send` inside a process, and `send` and `fast_send` in both directions between CPU and FPGA actors, with ordering, error reporting and the `fast_send` port’s priority over the mailbox checked by unit tests. Its latencies are not yet measured. The per-part cycle counts will come from high-level synthesis, and the PCIe component from measurement on a card; we do not report either here.

10 Related work

Actor semantics. The operational semantics of Agha, Mason, Smith and Talcott [2] and the four-family semantics of De Koster and De Meuter [6] contain only asynchronous communication. We extend the latter. Our isolation and liveness results are stated against their isolated turn principle.

Synchronous calls. ABCL/1 [13] and ABS [9] provide synchronous calls between concurrent objects that block the caller while the callee runs on its own processor, and ABS adds reentrant synchronous calls inside a cog. E [11] restricts immediate calls to one vat. Extended Rebeca [12] adds synchronous message passing to the Rebeca actor language with a formal semantics; comparing its scheduling of synchronous messages with `fast_send` is left for future work. The survey of active-object languages by De Boer et al. [5] maps this design space.

Encodings. Honda and Tokoro [8] and Boudol [4] encode synchronous in asynchronous communication; Proposition 13 locates where such encodings stop applying once resources are bounded.

Actors on hardware. CAL and the StreamBlocks compiler [3] compile dataflow actors to both CPUs and FPGAs. Dataflow actors fire on token availability and have no synchronous call; our FPGA runtime targets the message-passing actor model with both primitives.

11 Limitations and future work

- The calculus and proofs are on paper. A mechanisation, for instance by extending the PLT Redex models of [6], would check them.
- The calculus has no failures; a `fast_send` to an actor on a failed resource waits forever. Timeouts and their interaction with isolation are open.
- Detecting cross-thread cycles at run time needs shared state (Proposition 11); its cost on the hot path is not measured.
- FPGA latencies await synthesis and a card; the GPU runtime is a design.
- Extended Rebeca’s synchronous messages should be compared with `fast_send` in detail.

12 Conclusion

`fast_send` adds a second primitive to the actor model: the receiver’s turn runs now, on a resource the caller waits on, and returns a value. Formally, it keeps the model’s isolation, makes determinism relative to replies, and replaces unconditional liveness with liveness exactly when waits are acyclic; asynchronous send alone is the acyclic case. It is not reducible to send-then-wait once execution resources are shared. A single global queue remains a sound simulator with `fast_send`, so a backtest or a debugger that runs every actor, in any language and on any substrate, from one queue shows only behaviour the real system can produce, though not all of it. And on every substrate we have built, `send` and `fast_send` are two different mechanisms — a queue, and an inline or blocking call — which is the practical case for treating both as primitives of the model.

References

- [1] G. Agha. *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press, 1986.
- [2] G. A. Agha, I. A. Mason, S. F. Smith, and C. L. Talcott. A foundation for actor computation. *Journal of Functional Programming*, 7(1):1–72, 1997.
- [3] E. Bezati et al. StreamBlocks: A compiler for heterogeneous dataflow computing. Technical report, EPFL, 2021.
- [4] G. Boudol. Asynchrony and the π -calculus. Technical Report 1702, INRIA Sophia-Antipolis, 1992.
- [5] F. de Boer, V. Serbanescu, R. Hähnle, L. Henrio, J. Rochas, C. C. Din, E. B. Johnsen, M. Sirjani, E. Khamespanah, K. Fernandez-Reyes, and A. M. Yang. A survey of active object languages. *ACM Computing Surveys*, 50(5), 2017.
- [6] J. De Koster and W. De Meuter. A formal specification for half a century of actor systems. Manuscript, Vrije Universiteit Brussel, 2023.
- [7] C. Hewitt, P. Bishop, and R. Steiger. A universal modular ACTOR formalism for artificial intelligence. In *IJCAI*, 1973.
- [8] K. Honda and M. Tokoro. An object calculus for asynchronous communication. In *ECOOP ’91*, LNCS 512, pages 133–147, 1991.
- [9] E. B. Johnsen, R. Hähnle, J. Schäfer, R. Schlatte, and M. Steffen. ABS: A core language for abstract behavioral specification. In *Formal Methods for Components and Objects (FMCO 2010)*, LNCS 6957, pages 142–164, 2012.
- [10] V. Mayeski. Evaluating suitability of the actor model of concurrency for high-frequency trading: the `fast_send` mechanism and a tick-to-book latency study. arXiv:2609.21173, 2026.
- [11] M. S. Miller, E. D. Tribble, and J. Shapiro. Concurrency among strangers: programming in E as plan coordination. In *Trustworthy Global Computing*, 2005.
- [12] M. Sirjani, F. S. de Boer, A. Movaghar, and A. Shali. Extended Rebeca: a component-based actor language with synchronous message passing. In *Application of Concurrency to System Design (ACSD)*, 2005.

- [13] A. Yonezawa, J.-P. Briot, and E. Shibayama. Object-oriented concurrent programming in ABCL/1. In *OOPSLA '86*, pages 258–268, 1986.